

This is exactly how you should approach a **complex distributed systems project**. Breaking it down into sequential phases ensures you are never overwhelmed and that **each piece is independently testable before you wire them all together**.

Here is your master project plan, structured sequentially from the foundational data layer all the way up to the user interface.

Phase 1: The Foundation (Database & Core OOP Refactoring)



Before we can build the API or the frontend, your core C++ logic needs a permanent home for its data. In this phase, we transition your in-memory vectors to persistent storage.

1. **Set up PostgreSQL:** Install PostgreSQL and design your database schema (Tables: Planets, Travellers, Tickets, SpaceTravels).
2. **Integrate libpqxx:** Add the PostgreSQL C++ driver to your project.
3. **Refactor OOP State:** Modify your Ticket, Traveller, and SpaceTravel classes. Remove the static vectors (like `pendingTicket` and `all_travellers`).
4. **Implement CRUD Operations:** Update your class methods (or create a data access layer) so that creating a user or a ticket executes SQL INSERT and SELECT queries instead of pushing to vectors.

Phase 2: The Asynchronous Engine (Message Queue & Worker)

Next, we build the background worker. This is the executable that will actually process the business logic in a scalable way.

1. **Set up RabbitMQ:** Install the RabbitMQ server to act as your message broker.
2. **Build the Worker Executable:** Create a new, separate C++ program (your "worker") that constantly listens to a specific RabbitMQ channel.
3. **Message Hydration:** Program the worker to parse incoming JSON payloads (containing `traveller_id`, `source_planet`, etc.).
4. **OOP Execution:** Have the worker use the parsed IDs to query PostgreSQL, instantiate your OOP classes (Planet, Astronaut, Ticket), and run your pricing math ($\$K / \text{time_to_travel}$) and validation rules.
5. **Final Commit:** If validation passes, the worker commits the final transaction to the database.

Phase 3: The Front Door & Traffic Cop (API Gateway & Cache)

Now we need to expose your backend to the web and protect it from race conditions (the "Thundering Herd" problem).

1. **Set up Drogon Framework:** Initialize a high-performance C++ Drogon HTTP server to act as your API Gateway.
2. **Create REST Endpoints:** Define routes like `POST /api/v1/tickets/book-intent`.
3. **Integrate Redis:** Connect your Drogon API to Redis using a C++ client like `redis-plus-plus`.
4. **Implement Distributed Locking:** When a booking request hits the API, immediately execute a `SETNX` (Set if Not Exists) command in Redis with a 5-minute TTL to lock the

specific seat.

5. **Queue Publishing:** If the lock is acquired, format a JSON message and push it to your RabbitMQ exchange, then return an HTTP 202 Accepted response to the client.

Phase 4: The User Interface (Frontend Development)

With the backend completely decoupled and secure, you can build the face of the application.

1. **Initialize React:** Set up a React project using TypeScript and Tailwind CSS for styling.
2. **Build the Booking Interface:** Create a UI where users can select their source, destination, and dates.
3. **Connect to API:** Write fetch calls to send the structured JSON payload to your Drogon API's endpoints.
4. **Handle UI States:** Program the frontend to handle success ("Booking Confirmed!"), pending ("Processing Confirmation"), and failure states ("Seat Temporarily Held by Another Passenger") based on the API responses.

Phase 5: Integration & Load Testing

The final step is proving your architecture works under pressure.

1. **End-to-End Walkthrough:** Book a ticket through the React UI and watch it flow through Drogon -> Redis -> RabbitMQ -> OOP Worker -> PostgreSQL.
2. **Thundering Herd Simulation:** Write a simple script (e.g., in Python or Bash) to fire 100 simultaneous HTTP requests at the same seat.
3. **Verify the Architecture:** Confirm that Redis successfully blocks 99 of the requests, RabbitMQ smoothly queues the single successful one, and PostgreSQL only records one final transaction.

System Master Sequence Document

1. Infrastructure & Environment Setup

- [] Install PostgreSQL & design schema.
- [] Install Redis.
- [] Install RabbitMQ.
- [] Configure C++ build system (CMake recommended) for libraries: libpqxx, redis-plus-plus, librdkafka (or equivalent AMQP client), and Drogon.

2. Core OOP Refactoring (C++)

- [] Replace static vector storage with database connection logic.
- [] Update Ticket, Traveller (and subclasses), and SpaceTravel constructors to hydrate from database rows.
- [] Ensure encapsulated logic (pricing, capacity checks) remains intact.

3. Background Worker Engine (C++)

- [] Create standalone worker executable.
- [] Implement RabbitMQ message consumption loop.
- [] Implement logic to deserialize JSON message -> Instantiate OOP Objects -> Validate -> Execute SQL INSERT/UPDATE via libpqxx.

- ☐ Implement logic to send a delete command to Redis upon successful DB commit.

4. API Gateway (C++ Drogon)

- ☐ Initialize Drogon server.
- ☐ Create POST endpoint for booking intent.
- ☐ Implement Redis SETNX lock check (reject with HTTP 409 if locked).
- ☐ Implement RabbitMQ message publishing.
- ☐ Return HTTP 202 success response.

5. Frontend App (React/TS)

- ☐ Scaffold React app with Tailwind CSS.
- ☐ Build booking form (Source, Destination, Date).
- ☐ Wire form submission to Drogon API endpoint.
- ☐ Build UI feedback for lock rejection and processing states.